Systems Foundations

Deep Dive into DeepSeek — Chapter 19

Yin Yang

Foundation Books Project

The one rule of this chapter

A model can compute the right next-token distribution
and a serving system can still mishandle the request.

- The request may wait behind incompatible work, lose the cache state its next position needs, write into a page owned by a different sequence, or expose output before the producing device work has finished.
- None of those failures changes the Transformer equation. They arise because a working system must preserve the request while its representation, owner, and readiness change underneath it.

The one rule of this chapter

A model can compute the right next-token distribution
and a serving system can still mishandle the request.

- The request may wait behind incompatible work, lose the cache state its next position needs, write into a page owned by a different sequence, or expose output before the producing device work has finished.
- None of those failures changes the Transformer equation. They arise because a working system must preserve the request while its representation, owner, and readiness change underneath it.

Goal: build one reusable vocabulary — identity, ownership, completion, evidence — and follow it through CUDA execution, fabric movement, distributed ownership, and pipeline scheduling.

Roadmap

1. One request, several simultaneous views

Roadmap

1. One request, several simultaneous views
2. Execution coordinates, correctness, and V3 co-design

Roadmap

1. One request, several simultaneous views
2. Execution coordinates, correctness, and V3 co-design
3. CUDA and Tensor Core execution

Roadmap

1. One request, several simultaneous views
2. Execution coordinates, correctness, and V3 co-design
3. CUDA and Tensor Core execution
4. Fabric, distributed ownership, and pipelines

Roadmap

1. One request, several simultaneous views
2. Execution coordinates, correctness, and V3 co-design
3. CUDA and Tensor Core execution
4. Fabric, distributed ownership, and pipelines
5. What carries forward

One request, several simultaneous views

The live request systems record

Definition (Live request systems record)
At systems step t, a live request systems record for request r is

$$\mathcal{S}_t(r) = \big(I_r, Q_t(r), W_t(r), C_t(r), O_t(r)\big).$$

- I_r: stable request identity and input contract.

The live request systems record

Definition (Live request systems record)
At systems step t, a live request systems record for request r is

$$\mathcal{S}_t(r) = \big(I_r, Q_t(r), W_t(r), C_t(r), O_t(r)\big).$$

- I_r: stable request identity and input contract.
- $Q_t(r)$: admission, waiting, running, or terminal state.

The live request systems record

Definition (Live request systems record)
At systems step t, a live request systems record for request r is

$$\mathcal{S}_t(r) = \big(I_r, Q_t(r), W_t(r), C_t(r), O_t(r)\big).$$

- I_r: stable request identity and input contract.
- $Q_t(r)$: admission, waiting, running, or terminal state.
- $W_t(r)$: work selected for the current model step.

The live request systems record

Definition (Live request systems record)
At systems step t, a live request systems record for request r is

$$\mathcal{S}_t(r) = \big(I_r, Q_t(r), W_t(r), C_t(r), O_t(r)\big).$$

- I_r: stable request identity and input contract.
- $Q_t(r)$: admission, waiting, running, or terminal state.
- $W_t(r)$: work selected for the current model step.
- $C_t(r)$: resident model state plus its mapping and ownership.

The live request systems record

Definition (Live request systems record)
At systems step t, a live request systems record for request r is

$$\mathcal{S}_t(r) = \big(I_r, Q_t(r), W_t(r), C_t(r), O_t(r)\big).$$

- I_r: stable request identity and input contract.
- $Q_t(r)$: admission, waiting, running, or terminal state.
- $W_t(r)$: work selected for the current model step.
- $C_t(r)$: resident model state plus its mapping and ownership.
- $O_t(r)$: the remaining output and completion obligation.

The live request systems record

Definition (Live request systems record)
At systems step t, a live request systems record for request r is

$$\mathcal{S}_t(r) = \big(I_r, Q_t(r), W_t(r), C_t(r), O_t(r)\big).$$

- I_r: stable request identity and input contract.
- $Q_t(r)$: admission, waiting, running, or terminal state.
- $W_t(r)$: work selected for the current model step.
- $C_t(r)$: resident model state plus its mapping and ownership.
- $O_t(r)$: the remaining output and completion obligation.

A component may transform one field, but must keep the result attached to the same request identity I_r.

Four systems distinctions that recur all chapter

Count	What it measures
Request count	requests admitted, waiting, or running

Four systems distinctions that recur all chapter

Count	What it measures
Request count	requests admitted, waiting, or running
Scheduled-row count	rows selected for the current model step

Four systems distinctions that recur all chapter

Count	What it measures
Request count	requests admitted, waiting, or running
Scheduled-row count	rows selected for the current model step
Resident-position count	earlier positions attention may still read

Four systems distinctions that recur all chapter

Count	What it measures
Request count	requests admitted, waiting, or running
Scheduled-row count	rows selected for the current model step
Resident-position count	earlier positions attention may still read
Allocated-slot count	physical page slots currently owned

Four systems distinctions that recur all chapter

Count	What it measures
Request count	requests admitted, waiting, or running
Scheduled-row count	rows selected for the current model step
Resident-position count	earlier positions attention may still read
Allocated-slot count	physical page slots currently owned

Invariant (Systems-account separation)
These four counts must remain separate. A conversion between two counts must name the request phase, cache representation, allocation geometry, and scope of the account.

Four systems distinctions that recur all chapter

Count	What it measures
Request count	requests admitted, waiting, or running
Scheduled-row count	rows selected for the current model step
Resident-position count	earlier positions attention may still read
Allocated-slot count	physical page slots currently owned

Invariant (Systems-account separation)
These four counts must remain separate. A conversion between two counts must name the request phase, cache representation, allocation geometry, and scope of the account.

Issue is not completion: a producer's issued work becomes safe input only after its matching completion.

Execution coordinates, correctness, and V3 co-design

Same shape, different executable

$$\mathcal{V} = (r, c, p, e)$$

- r: repository. c: exact commit. p: built package or container. e: hardware/software environment that selected the executed path.
- Two calls with the same tensor shape can execute different code: a changed revision may select another backend, a changed environment may select another specialization.

Same shape, different executable

$$\mathcal{V} = (r, c, p, e)$$

- r: repository. c: exact commit. p: built package or container. e: hardware/software environment that selected the executed path.
- Two calls with the same tensor shape can execute different code: a changed revision may select another backend, a changed environment may select another specialization.

$\mathcal{V}$ identifies the executable environment. The measurement protocol separately fixes workload, oracle, and timing window.

Correctness before performance

Tempting check	Hidden failure it can miss
Shape and dtype match	scale grouping interpreted under the wrong owner

Correctness before performance

Tempting check	Hidden failure it can miss
Shape and dtype match	scale grouping interpreted under the wrong owner
Output tensor has right extent	combine uses a stale inverse route map

Correctness before performance

Tempting check	Hidden failure it can miss
Shape and dtype match	scale grouping interpreted under the wrong owner
Output tensor has right extent	combine uses a stale inverse route map
Fast path launched	its eligibility was stale or ineligible

Correctness before performance

Tempting check	Hidden failure it can miss
Shape and dtype match	scale grouping interpreted under the wrong owner
Output tensor has right extent	combine uses a stale inverse route map
Fast path launched	its eligibility was stale or ineligible
Producer issued successfully	consumer read before matching completion

Correctness before performance

Tempting check	Hidden failure it can miss
Shape and dtype match	scale grouping interpreted under the wrong owner
Output tensor has right extent	combine uses a stale inverse route map
Fast path launched	its eligibility was stale or ineligible
Producer issued successfully	consumer read before matching completion

A visible token or correctly shaped tensor can pass while the hidden state the next component needs is already wrong.

Pressure transfer, not a free lunch

Definition (Co-design ledger)
$\mathcal{L}_d = (d,\ o_{\text{before}} \to o_{\text{after}},\ p_{\downarrow}, p_{\uparrow}, q, \text{e})$: the object that changed, the pressure relieved, the pressure added, the new obligation, and the evidence class.

- MLA relieves retained KV traffic, adds latent projection and layout obligations.

Pressure transfer, not a free lunch

Definition (Co-design ledger)
$\mathcal{L}_d = (d,\ o_{\text{before}} \to o_{\text{after}},\ p_{\downarrow}, p_{\uparrow}, q, \mathrm{e})$: the object that changed, the pressure relieved, the pressure added, the new obligation, and the evidence class.

- MLA relieves retained KV traffic, adds latent projection and layout obligations.
- DeepSeekMoE relieves dense expert arithmetic, adds routing, dispatch, and placement metadata.

Pressure transfer, not a free lunch

Definition (Co-design ledger)
$\mathcal{L}_d = (d,\ o_{\text{before}} \to o_{\text{after}},\ p_{\downarrow}, p_{\uparrow}, q, \mathrm{e})$: the object that changed, the pressure relieved, the pressure added, the new obligation, and the evidence class.

- MLA relieves retained KV traffic, adds latent projection and layout obligations.
- DeepSeekMoE relieves dense expert arithmetic, adds routing, dispatch, and placement metadata.
- FP8 relieves payload width, adds scale metadata and accumulation rules.

Pressure transfer, not a free lunch

Definition (Co-design ledger)
$\mathcal{L}_d = (d,\ o_{\text{before}} \to o_{\text{after}},\ p_{\downarrow}, p_{\uparrow}, q, \mathrm{e})$: the object that changed, the pressure relieved, the pressure added, the new obligation, and the evidence class.

- MLA relieves retained KV traffic, adds latent projection and layout obligations.
- DeepSeekMoE relieves dense expert arithmetic, adds routing, dispatch, and placement metadata.
- FP8 relieves payload width, adds scale metadata and accumulation rules.
- DualPipe and co-designed all-to-all relieve exposed pipeline time, add dependency tracking and resource competition.

Pressure transfer, not a free lunch

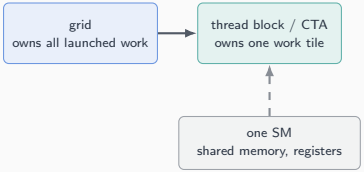
Definition (Co-design ledger)
$\mathcal{L}_d = (d,\ o_{\text{before}} \to o_{\text{after}},\ p_{\downarrow}, p_{\uparrow}, q, \text{e})$: the object that changed, the pressure relieved, the pressure added, the new obligation, and the evidence class.

- MLA relieves retained KV traffic, adds latent projection and layout obligations.
- DeepSeekMoE relieves dense expert arithmetic, adds routing, dispatch, and placement metadata.
- FP8 relieves payload width, adds scale metadata and accumulation rules.
- DualPipe and co-designed all-to-all relieve exposed pipeline time, add dependency tracking and resource competition.

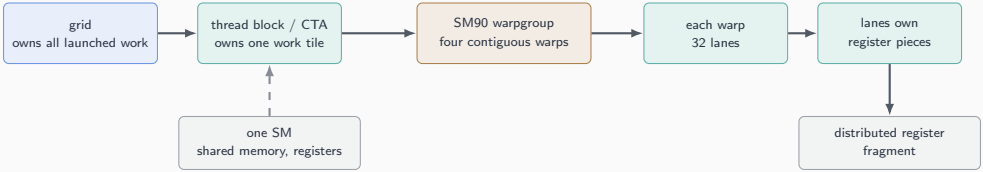
[REPORTED: DeepSeek-V3 technical report]: relieved pressure and added obligation are reported together — never one without the other.

CUDA and Tensor Core execution

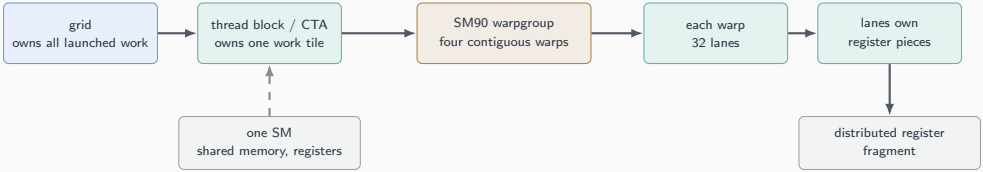
From grid to lane: who owns what



From grid to lane: who owns what



From grid to lane: who owns what



A CTA owns a work tile; a warpgroup names participating threads; a fragment names distributed per-lane state — three different nouns for three different things.

From concept to code: WGMMA's three operation-group boundaries

from DeepGEMM/deep_gemm/include/deep_gemm/ptx/wgmma.cuh

```
CUTLASS_DEVICE void warpgroup_arrive() {
    asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory");
}
CUTLASS_DEVICE void warpgroup_commit_batch() {
    asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory");
}
template <int N>
CUTLASS_DEVICE void warpgroup_wait() {
    asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(N) : "memory");
}
```

From concept to code: WGMMA's three operation-group boundaries

from DeepGEMM/deep_gemm/include/deep_gemm/ptx/wgmma.cuh

```
CUTLASS_DEVICE void warpgroup_arrive() {
    asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory");
}
CUTLASS_DEVICE void warpgroup_commit_batch() {
    asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory");
}
template <int N>
CUTLASS_DEVICE void warpgroup_wait() {
    asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(N) : "memory");
}
```

Fence orders register access; commit closes a batch into a tracked group; wait drains pending groups before their accumulator is read. None of the three is the matrix operation itself.

[INSPECTED: DeepGEMM ptx/wgmma.cuh, revision 1f6f3f378920]

Two asynchronous engines, two completion facts

- TMA moves a tile between global and shared memory; a transaction-backed `mbarrier` proves the promised bytes arrived.

Two asynchronous engines, two completion facts

- TMA moves a tile between global and shared memory; a transaction-backed `mbarrier` proves the promised bytes arrived.
- WGMMA (SM90) or `tcgen05.mma` (SM100) accumulates a matrix update; its own commit/wait or completion protocol proves the result is safe to read.

Two asynchronous engines, two completion facts

- TMA moves a tile between global and shared memory; a transaction-backed `mbarrier` proves the promised bytes arrived.
- WGMMA (SM90) or `tcgen05.mma` (SM100) accumulates a matrix update; its own commit/wait or completion protocol proves the result is safe to read.
- TMEM (SM100) is dedicated on-chip matrix storage — a different surface from ordinary shared memory or TMA's transfer path.

Two asynchronous engines, two completion facts

- TMA moves a tile between global and shared memory; a transaction-backed `mbarrier` proves the promised bytes arrived.
- WGMMA (SM90) or `tcgen05.mma` (SM100) accumulates a matrix update; its own commit/wait or completion protocol proves the result is safe to read.
- TMEM (SM100) is dedicated on-chip matrix storage — a different surface from ordinary shared memory or TMA's transfer path.

Copy completion and matrix completion are never the same fact. A reusable stage needs both, plus the next owner's handoff, before reuse.

Fabric, distributed ownership, and pipelines

RDMA and the same five-field handoff

- RDMA names a remote-memory access property: an initiator reads or writes eligible *registered* memory on another machine.

RDMA and the same five-field handoff

- RDMA names a remote-memory access property: an initiator reads or writes eligible *registered* memory on another machine.
- Posting a write places it on the path. It does not by itself prove the destination is safe to read — that comes from a completion or acknowledgment.

RDMA and the same five-field handoff

- RDMA names a remote-memory access property: an initiator reads or writes eligible *registered* memory on another machine.
- Posting a write places it on the path. It does not by itself prove the destination is safe to read — that comes from a completion or acknowledgment.
- A *rank* is a participant identifier inside one group; node, device, process, worker, and rank name different facts even when one toy example maps them one-to-one.

RDMA and the same five-field handoff

- RDMA names a remote-memory access property: an initiator reads or writes eligible *registered* memory on another machine.
- Posting a write places it on the path. It does not by itself prove the destination is safe to read — that comes from a completion or acknowledgment.
- A *rank* is a participant identifier inside one group; node, device, process, worker, and rank name different facts even when one toy example maps them one-to-one.

Fabric words specialize the same handoff: object, interpretation, owner, completion, scope — now with participants, eligible buffers, and a local-or-cross-node path.

Pipeline bubbles are idle slots, not slow work

- A *pipeline dependency*: a later stage waits for an incoming activation; an earlier stage waits for the later stage's gradient during backward.

Pipeline bubbles are idle slots, not slow work

- A *pipeline dependency*: a later stage waits for an incoming activation; an earlier stage waits for the later stage's gradient during backward.
- A *bubble* is an idle stage-time position in the chosen schedule — not a synonym for unfinished communication or slow computation.

Pipeline bubbles are idle slots, not slow work

- A *pipeline dependency*: a later stage waits for an incoming activation; an earlier stage waits for the later stage's gradient during backward.
- A *bubble* is an idle stage-time position in the chosen schedule — not a synonym for unfinished communication or slow computation.
- Two work items may overlap only when neither consumes state the other has not published, and their owners do not need the same exclusive resource.

Pipeline bubbles are idle slots, not slow work

- A *pipeline dependency*: a later stage waits for an incoming activation; an earlier stage waits for the later stage's gradient during backward.
- A *bubble* is an idle stage-time position in the chosen schedule — not a synonym for unfinished communication or slow computation.
- Two work items may overlap only when neither consumes state the other has not published, and their owners do not need the same exclusive resource.

DualPipe (Volume II) specializes this rule to bidirectional work, explicit transfers, and deferred weight-gradient actions.

What carries forward

Concept-to-code map for the chapter

- **launch record, ownership, CuTE roles** → `DeepSeek-V3/inference/model.py`

Concept-to-code map for the chapter

- **launch record, ownership, CuTE roles** → `DeepSeek-V3/inference/model.py`
- **TMA copy and fence wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`

Concept-to-code map for the chapter

- **launch record, ownership, CuTE roles** → `DeepSeek-V3/inference/model.py`
- **TMA copy and fence wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **transaction-backed mbarrier wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`

Concept-to-code map for the chapter

- **launch record, ownership, CuTE roles** → `DeepSeek-V3/inference/model.py`
- **TMA copy and fence wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **transaction-backed mbarrier wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **routed-row dispatch and combine** → `DeepEP/README.md`

Concept-to-code map for the chapter

- **launch record, ownership, CuTE roles** → `DeepSeek-V3/inference/model.py`
- **TMA copy and fence wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **transaction-backed mbarrier wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **routed-row dispatch and combine** → `DeepEP/README.md`
- **RDMA storage and chain replication** → `3FS/README.md`

Concept-to-code map for the chapter

- **launch record, ownership, CuTE roles** → `DeepSeek-V3/inference/model.py`
- **TMA copy and fence wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **transaction-backed mbarrier wrappers** → `DeepGEMM/deep_gemm/include/deep_gemm/ptx/tma.cuh`
- **routed-row dispatch and combine** → `DeepEP/README.md`
- **RDMA storage and chain replication** → `3FS/README.md`

Identity, ownership, completion, evidence — one vocabulary,
reused across CUDA, fabric, distributed systems, and pipelines.

Next: CUDA kernels, fabric movement, and pipeline scheduling each specialize this same handoff with their own concrete objects (Volume II).